

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	3
2.1 Namespace List	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 untangle Namespace Reference	9
5.1.1 Detailed Description	9
6 Class Documentation	11
6.1 untangle::executor Class Reference	11
6.1.1 Detailed Description	11
6.1.2 Member Typedef Documentation	12
6.1.2.1 task_t	12
6.1.3 Member Function Documentation	12
6.1.3.1 submit()	12
7 File Documentation	13
7.1 executor.hpp	13
Index	15

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>

int main() {
    untangle::executor pool(4);

    for (int i = 0; i < 100; ++i) {
        pool.submit([i] { work(i); });
    }

    return 0; // ~executor drains what was submitted, then stops the workers
}
```

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue drained — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `submit()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

lifetime and failure

The destructor drains. It refuses further work, waits for the queue to empty and for every worker to go idle, and only then stops them. A task submitted before the pool goes out of scope has run by the time it does.

`submit()` says whether the task was taken. It returns false once the pool is shutting down, which a task submitting more work from inside the pool can see.

A task that throws does not take the worker with it. `execution` catches it, reports it on `stderr`, and the worker carries on with the next task. Nothing reaches the submitter — a task that has something to report carries its own channel, because a continuous worker does not collect results.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are `address`, `thread`, `undefined`, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs `doxygen`, `tectonic` and `python3`, and it fails on any `doxygen` warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	9
--------------------------	---	-------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor	Runs tasks on a fixed pool of <code>untangle::async::execution</code> workers	11
------------------------------------	---	--------------------

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	13
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

Imported from the `async` repo's `prototypes/executor.hpp`, unchanged apart from its namespace and this comment. The hardening the prototype's README asks for has not been done yet.

The pool holds `N` executions in continuous mode and a shared queue of tasks. A task is handed straight to a free worker only when the shared queue is empty; otherwise it goes to the back of the queue, and a worker takes the front of the queue as it frees up. Nothing overtakes: finding a free worker does not let a task jump a queue that already has work in it.

Chapter 6

Class Documentation

6.1 untangle::executor Class Reference

Runs tasks on a fixed pool of untangle::async::execution workers.

```
#include <executor.hpp>
```

Public Types

- using `task_t` = `std::function<void(void)>`

Public Member Functions

- `executor (std::size_t worker_count)`
Starts `worker_count` executions in continuous mode.
- `~executor ()`
Drains what has been submitted, then stops every worker.
- `bool submit (task_t task)`
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending () const`
How many tasks are waiting for a worker.

6.1.1 Detailed Description

Runs tasks on a fixed pool of untangle::async::execution workers.

Remarks

Tasks are `std::function<void(void)>`. An execution's action type is fixed by its template argument, so one pool runs one signature; a task that needs to return something carries its own channel, because a continuous worker does not collect results.

6.1.2 Member Typedef Documentation

6.1.2.1 task_t

```
using untangle::executor::task_t = std::function<void(void)>
```

The work a pool runs. One pool runs one signature, because an execution's action type is fixed by its template argument.

6.1.3 Member Function Documentation

6.1.3.1 submit()

```
bool untangle::executor::submit (  
    task_t task) [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Returns

true - the task was accepted. false - the pool is shutting down and refused it.

The documentation for this class was generated from the following file:

- executor.hpp

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00014 #pragma once
00015
00016 #include <async.hpp>
00017 #include <condition_variable>
00018 #include <cstdint>
00019 #include <deque>
00020 #include <functional>
00021 #include <memory>
00022 #include <mutex>
00023 #include <string>
00024 #include <utility>
00025 #include <vector>
00026
00027 namespace untangle {
00028
00036 class executor {
00037 public:
00040     using task_t = std::function<void(void)>;
00041
00045     explicit executor(std::size_t worker_count) {
00046         workers_.reserve(worker_count);
00047
00048         for (std::size_t index = 0; index < worker_count; ++index) {
00049             worker next;
00050             next.exec = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00051
00052             // How a worker learns there is more waiting for it. is_busy() answers whether a worker has
00053             // room; this is what tells the pool the moment one frees up, without anybody polling.
00054             next.exec->on_finished = [this, index] { take_next_task(index); };
00055
00056             workers_.push_back(std::move(next));
00057             workers_.back().exec->start();
00058         }
00059     }
00060
00064     ~executor() {
00065     {
00066         std::unique_lock<std::mutex> lock(mutex_);
00067         accepting_ = false;
00068         drained_cv_.wait(lock, [this] { return pending_.empty() && nothing_running(); });
00069     }
00070
00071     // Outside the lock: stop() wakes each worker, and a worker waking up runs on_finished, which
00072     // wants this mutex.
00073     for (auto& one : workers_) {
00074         one.exec->stop();
00075     }
00076
00077     // Releasing the last shared_ptr runs ~execution(), which waits for the detached worker.
00078     workers_.clear();
00079 }
00080
00086 bool submit(task_t task) {
00087     std::lock_guard<std::mutex> lock(mutex_);
00088
```

```

00089     if (!accepting_) {
00090         return false;
00091     }
00092
00093     // The queue comes first whenever it has anything in it: a task that arrives while others are
00094     // waiting does not get to overtake them just because a worker happens to be free.
00095     if (pending_.empty()) {
00096         for (std::size_t index = 0; index < workers_.size(); ++index) {
00097             if (!workers_[index].exec->is_busy()) {
00098                 give_to_worker(index, std::move(task));
00099                 return true;
00100             }
00101         }
00102     }
00103
00104     pending_.push_back(std::move(task));
00105     return true;
00106 }
00107
00111 std::size_t pending() const {
00112     std::lock_guard<std::mutex> lock(mutex_);
00113     return pending_.size();
00114 }
00115
00116 private:
00117     using worker_execution = untangle::async::execution<std::function<void(void)>>;
00118
00119     struct worker {
00120         std::shared_ptr<worker_execution> exec;
00121     };
00122
00129 void give_to_worker(std::size_t index, task_t task) {
00130     // add_action() takes the execution's own action_mutex while this holds mutex_. That is only
00131     // safe in one direction, and it holds: a worker calls on_finished with action_mutex released,
00132     // so it never takes mutex_ while holding action_mutex, and the two never form a cycle.
00133     workers_[index].exec->add_action(std::move(task));
00134 }
00135
00139 void take_next_task(std::size_t index) {
00140     std::lock_guard<std::mutex> lock(mutex_);
00141
00142     if (!pending_.empty()) {
00143         task_t next = std::move(pending_.front());
00144         pending_.pop_front();
00145         give_to_worker(index, std::move(next));
00146         return;
00147     }
00148
00149     if (nothing_running()) {
00150         drained_cv_.notify_all();
00151     }
00152 }
00153
00160 bool nothing_running() const {
00161     for (const auto& one : workers_) {
00162         if (one.exec->is_busy()) {
00163             return false;
00164         }
00165     }
00166
00167     return true;
00168 }
00169
00170 mutable std::mutex mutex_;
00171 std::condition_variable drained_cv_;
00172
00173 std::deque<task_t> pending_;
00174 std::vector<worker> workers_;
00175 bool accepting_ = true;
00176 };
00177
00178 } // namespace untangle

```

Index

executor, [1](#)

submit

 untangle::executor, [12](#)

task_t

 untangle::executor, [12](#)

untangle, [9](#)

untangle::executor, [11](#)

 submit, [12](#)

 task_t, [12](#)